

LLM Virtual Lab

Complete Syllabus Reference

4 Weeks • 4 Modules • 15 Experiments

Aims • ELI5 Theory • Flows • Tables • Image Prompts

Designed so a primary school student can understand it — simple language, visual flows, zero jargon overload.

Table of Contents

W1 Exp 1	Tokenization and Text Encoding (BPE)
W1 Exp 2	Word Embeddings and Vector Representations
W1 Exp 3	Exploring the Transformer Architecture
W1 Exp 4	Understanding Self-Attention
W2 Exp 1	Zero-Shot Prompting
W2 Exp 2	Few-Shot Prompting
W2 Exp 3	Chain-of-Thought (CoT) Prompting
W2 Exp 4	Structured Output Generation
W3 Exp 1	Building a Chatbot
W3 Exp 2	AI-Based Text Summarization
W3 Exp 3	Document Question Answering
W4 Exp 1	Creating Embeddings
W4 Exp 2	Semantic Search
W4 Exp 3	Building a Simple RAG Pipeline
W4 Exp 4	Fine-Tuning vs Prompt Engineering

Week 1 | Module 1: Foundations of LLMs

Lab 1: Understanding Language Models

Experiment 1: Tokenization and Text Encoding (BPE)

■ AIM

To implement a Byte-Pair Encoding tokenization pipeline from first principles by iteratively merging the most frequent character pairs in a corpus, building a subword vocabulary that efficiently balances coverage and compression across diverse text inputs.

■ THEORY

The Two Old Problems

Computers used to read text in two ways — both had big problems:

Approach	What It Does	The Problem
Whole-Word	Looks up each word in a giant dictionary	New words / typos = total failure (OOV error)
Letter-by-Letter	Reads one character at a time	Sentences become too long — computer forgets the start by the end

BPE — The Goldilocks Fix

BPE starts with letters, then keeps merging the most popular pairs into new symbols — like inventing shorthand for common combinations.

Start: u n p r e d i c t a b l e	→	Find most frequent pair	→	Merge it into one new symbol	→	Repeat th ousands of times	→	Result: un + predict + able
--	---	----------------------------------	---	---------------------------------------	---	----------------------------------	---	--------------------------------------

Why BPE wins:

Feature	BPE Behaviour
Common words	Single token — 'the' = 1 piece
Rare / new words	Split into chunks — 'un' + 'predict' + 'able'
Unknown words	Never crashes — always finds pieces
GPT-4 upgrade	Starts from 256 bytes — works for ALL languages

■ IMAGE PROMPT

Soft pastel infographic: the word 'unpredictable' split into LEGO blocks labeled 'un', 'predict', 'able' in mint green, peach, lavender; small robot watching; arrow flow showing merging steps; minimal text; light cream background.

Experiment 2: Word Embeddings and Vector Representations

■ AIM

To generate dense vector representations of words and sentences using pre-trained embedding models, visualize their positions in high-dimensional semantic space, and quantify the conceptual similarity between text pairs using Cosine Similarity as the core distance metric.

■ THEORY

From IDs to a Map

Old method gave every word a unique number — but Cat=1 and Dog=2 tells the computer nothing about their relationship. Embeddings fix this by placing every word on a giant invisible map. Words used in similar sentences end up near each other.

Method	Knows Cat ≈ Dog?	Can do King–Man+Woman=Queen?
One-Hot ID	No	No
Word Embeddings	Yes — they are close on the map	Yes — directions carry meaning

Cosine Similarity — the Angle Trick

We measure closeness by the angle between two word-arrows, not their length.

Angle	Score	Meaning
0°	1.0	Same meaning
90°	0.0	Unrelated
180°	-1.0	Opposite meaning

■ IMAGE PROMPT

Soft pastel 2D grid like a treasure map; coloured dots for cat, dog, king, queen; dotted lines between similar words; protractor showing cosine angle; mint and peach palette; minimal text.

Experiment 3: Exploring the Transformer Architecture

■ AIM

To dissect and map the full structural anatomy of the original Encoder-Decoder Transformer architecture, and implement its Positional Encoding layer from scratch using alternating sine and cosine wave functions to understand how sequence order is preserved without recurrence.

■ THEORY

The Old Way (RNN) vs The New Way (Transformer)

	RNN / LSTM	Transformer
Reads text	One word at a time	All words simultaneously
Speed	Slow — sequential	Fast — fully parallel
Memory	Forgets early words in long text	Every word can see every other word
GPU friendly?	No	Yes

The Ordering Problem & Positional Encoding

Reading everything at once loses word order — 'Dog bites man' and 'Man bites dog' look identical. Fix: stamp each word with a unique wave signature before processing.

Tokens in	→	Add wave stamp (PE)	→	Now each word has a seat number	→	Encoder reads all at once	→	Meaning preserved ✓
-----------	---	---------------------	---	---------------------------------	---	---------------------------	---	---------------------

Encoder vs Decoder — Quick Map

Part	Job	Example
Encoder	Reads & understands input	Reading English
Decoder	Generates output word by word	Writing French
Both together	Full translation / summarisation	GPT, T5, BERT

■ IMAGE PROMPT

Soft pastel two-panel infographic: left — person reading scroll one word at a time (RNN, sad face); right — robot photographing entire sentence (Transformer, happy face); below: sine/cosine wave stamps on word bubbles; peach and light blue; minimal text; cream background.

Experiment 4: Understanding Self-Attention

■ AIM

To manually implement the complete Scaled Dot-Product Attention mechanism from first principles — computing Query, Key, and Value projections, applying scaled dot-product scoring, and using softmax normalization — to observe how each token dynamically routes and weights contextual information from every other token in the sequence.

■ THEORY

The Library Analogy — Q, K, V

Vector	Role	Library Analogy
Q — Query	What am I looking for?	Your search question
K — Key	What do I offer to others?	A book's title / label
V — Value	What is my actual content?	The book's inner knowledge

The Formula — Step by Step

Attention(Q, K, V) = softmax($QK^T / \sqrt{d_k}$) × V

Q × K Score every word pair	→	÷ √d Turn down the volume	→	softmax() Convert to percentages	→	× V Blend values by score	→	Context-rich output token
--------------------------------	---	------------------------------	---	-------------------------------------	---	---------------------------	---	---------------------------

Why the Scaling ÷ √d Matters

Without it, scores explode at high dimensions → softmax outputs near-zero gradients → the model stops learning. Dividing by √d keeps scores in a healthy range.

■ IMAGE PROMPT

Soft pastel: row of word bubbles with curved arrows of varying thickness showing attention scores; Q K V three-column table in mint/peach/lavender; softmax funnel turning numbers into percentages; cream background; minimal text; friendly rounded shapes.

Week 2 | Module 2: Prompt Engineering

Lab 2: Interacting with LLMs

Experiment 1: Zero-Shot Prompting

■ AIM

To evaluate the baseline generalization capabilities of a Large Language Model by designing and testing structured task prompts that contain no examples, measuring how accurately the model leverages its pre-trained knowledge to perform classification, translation, and instruction-following tasks under strict system-level behavioral constraints.

■ THEORY

What Is Zero-Shot?

You give the AI a task with zero examples. The AI uses only what it already learned from reading billions of internet pages during training.



System Prompt vs User Prompt

Layer	Who Sets It	What It Does
System Prompt	Developer	Sets the rulebook — persona, format, constraints
User Prompt	User	The actual question or task being asked

When Zero-Shot Works vs Fails

Task	Zero-Shot?
Translate to French	Works
Sentiment analysis	Works
Custom JSON schema	Often fails
Niche domain labels	Often fails

■ IMAGE PROMPT

Soft pastel: robot standing alone in front of chalkboard with question mark; human speech bubble 'translate to French'; robot pointing to answer with glowing brain; two-layer stacked box showing system/user prompt; cream background; minimal text.

Experiment 2: Few-Shot Prompting

■ AIM

To implement In-Context Learning by engineering a curated set of balanced input-output exemplars directly inside the prompt payload, guiding the model to replicate specific output patterns and classification schemas without modifying its underlying weights, while actively mitigating Majority Label Bias and Recency Bias through deliberate exemplar ordering.

■ THEORY

The Idea — Show, Don't Just Tell

Instead of explaining the rules, you paste a few perfect examples directly into the prompt. The AI copies the pattern — instantly, no retraining needed.

Example 1: 'Cat' → Animal	Example 2: 'Rose' → Plant	Example 3: 'Eagle' → Animal
New question: 'Oak' → ?	AI sees the pattern →	Outputs: Plant ✓

The Hidden Dangers — Bias

Bias Type	What Happens	The Fix
Majority Label Bias	3 Positive examples, 1 Negative → AI always says Positive	Balance labels equally
Recency Bias	AI blindly copies the pattern of the last example	Shuffle example order randomly

■ IMAGE PROMPT

Soft pastel: three neat example cards (input → output label); fourth card with question mark; robot copying the pattern with lightbulb; warning sign showing unbalanced cards (bias); mint green peach lavender palette; cream background; minimal text.

Experiment 3: Chain-of-Thought (CoT) Prompting

■ AIM

To improve LLM accuracy on complex multi-step reasoning, mathematical deduction, and logical inference tasks by prompting the model to generate explicit intermediate reasoning steps before producing a final answer, effectively expanding its working memory and reducing catastrophic hallucination rates.

■ THEORY

The Problem — No Scratch Paper

When you demand an instant final answer to a complex problem, the AI must compress 10 steps into 1 prediction — and usually gets it wrong. CoT gives it scratch paper to think on.

	Without CoT	With CoT
How AI answers	Guesses final answer in one shot	Writes every step, then answers
Accuracy on maths	Often wrong	Dramatically better
Hallucination risk	High	Low

How Each Step Feeds the Next

User asks hard question	→	AI writes Step 1	→	Step 1 added to context	→	AI writes Step 2...	→	Final answer is grounded ✓
-------------------------	---	------------------	---	-------------------------	---	---------------------	---	----------------------------

Two Ways to Trigger CoT

Method	How	Effort
Zero-Shot CoT	Append 'Let us think step by step'	One line — easy
Few-Shot CoT	Provide full worked examples with steps shown	More effort — more reliable

■ IMAGE PROMPT

Soft pastel: left side robot facing maths problem → single arrow → wrong answer (red cross); right side same robot with notepad writing step 1, 2, 3 → green checkmark; label 'no scratch paper vs thinking out loud'; light blue mint peach; cream background; minimal text.

Experiment 4: Structured Output Generation

■ AIM

To enforce strict, machine-parseable output formatting from an LLM by applying both prompt-level schema constraints and grammar-level token filtering techniques, ensuring every response conforms to a valid JSON or Pydantic schema structure suitable for direct programmatic ingestion by downstream APIs and databases.

■ THEORY

The Problem — AIs Love to Chat

By default the AI says: 'Sure! Here is the JSON: { ... } Hope this helps!' The word 'Sure' instantly crashes any program trying to parse that as JSON.

	Default AI Output	Structured Output
Format	Conversational prose + data	Pure valid JSON only
JSON.parse() result	CRASH	Works perfectly
Use in production?	No	Yes

Two Enforcement Levels

Method	How It Works	Reliability
Prompt Constraints	Tell the AI: output ONLY valid JSON, no markdown	~90% — AI can still slip
Grammar-Level Constraints	AI's engine blocks any token that violates the schema mathematically	100% — physically impossible to fail

■ IMAGE PROMPT

Soft pastel two-panel: left — robot outputting messy text bubble, red crash symbol, computer labeled 'JSON parse error'; right — robot inside schema box, clean key-value output, green checkmark; below: token gate blocking invalid words; mint peach lavender; cream background.

Week 3 | Module 3: Building AI Applications

Lab 3: LLM API Integration

Experiment 1: Building a Chatbot

■ AIM

To architect a fully stateful conversational interface by engineering a client-side rolling message history loop that appends every user and assistant turn with its role label, transmits the full history on every API call, and implements a truncation strategy — either Sliding Window or Summary Buffer — to sustain coherent long-term dialogue within the model's hard context window limit.

■ THEORY

Shocking Fact — The AI Has No Memory!

Every API call starts completely fresh. The AI instantly forgets everything after replying. ChatGPT only seems to remember because the app secretly sends the entire conversation history with every message.



The Three Roles in the History Array

Role	Who	Example
system	Developer	'You are a helpful assistant. Reply concisely.'
user	Human	'What is the capital of France?'
assistant	AI	'The capital of France is Paris.'

When History Gets Too Big — Truncation Strategies

Strategy	What It Does	Keeps Old Context?	Cost
Sliding Window	Delete the oldest messages when limit is near	No — gone forever	Low
Summary Buffer	Summarise old messages into one compact block	Yes — compressed	Medium

■ IMAGE PROMPT

Soft pastel: timeline of alternating user/assistant chat bubbles; bracket gathering all into one payload arrow to cloud API; sliding window deleting old bubbles on left; summariser robot compressing bubbles into one card; mint peach lavender; cream background.

Experiment 2: AI-Based Text Summarization

■ AIM

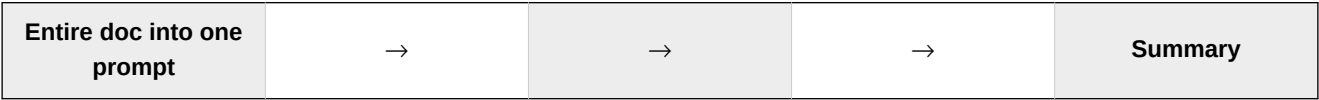
To design and benchmark three document summarization architectures — Stuffing, Map-Reduce, and Refine — against documents of varying sizes that exceed model context limits, comparing each strategy's output quality, context preservation depth, and total API call efficiency to determine the optimal approach for different document types.

■ THEORY

The Challenge — Document Won't Fit!

AI models can only read a limited number of words at once (context window). A 500-page legal contract is way too big. We need smart chunking strategies.

Strategy 1 — Stuffing



Strategy 2 — Map-Reduce



Strategy 3 — Refine



Strategy Comparison

Strategy	Speed	Context Quality	Best For
Stuffing	Instant	Perfect	Short docs only
Map-Reduce	Fast	Some loss	Large docs, speed matters
Refine	Slow	Excellent	Legal / technical docs

■ IMAGE PROMPT

Soft pastel three-row infographic: row 1 stuffing (one big doc → one box with overflow warning); row 2 map-reduce (doc split into 3 chunks each summarised then merged); row 3 refine (relay race baton passing summary forward); peach mint lavender; cream background.

Experiment 3: Document Question Answering

■ AIM

To build a context-grounded question answering pipeline that retrieves the most relevant passages from a provided document, injects them directly into a strictly constrained system prompt, and forces the LLM to extract verifiable answers exclusively from the supplied context — explicitly instructing the model to respond with 'I don't know' when the answer is absent — eliminating hallucination and creating a full audit trail from answer back to source.

■ THEORY

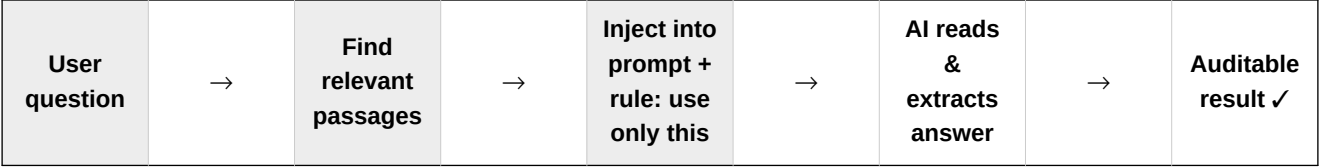
The Problem — AI Guesses From Memory

Ask an AI about your company's private HR policy and it invents a confident answer that sounds right but is completely made up — it was never trained on your document.

	Standard AI	Document QA
Knowledge source	Pre-trained memory	The document you provide

Hallucination risk	Very High	Very Low
Can be audited?	No	Yes — answer traces back to source
Unknown answer?	Invents one	Says 'I don't know'

The Extraction Flow



■ IMAGE PROMPT

Soft pastel: user with question bubble → document with magnifying glass highlighting passages → structured prompt box with lock icon 'only use this context' → robot outputting answer with chain link to source; side panel: robot saying 'I don't know' with thumbs up (safe behavior); mint peach lavender; cream background.

Week 4 | Module 4: Advanced LLM Systems

Lab 4: Retrieval-Augmented Generation

Experiment 1: Creating Embeddings

■ AIM

To build a document ingestion pipeline that systematically chunks raw text into semantically coherent blocks, converts each chunk into a dense high-dimensional vector using a pre-trained embedding model, applies L2 normalization to standardize all vector magnitudes to 1.0, and stores the resulting vectors in a structured format ready for similarity-based retrieval.

■ THEORY

From Words to Numbers to Meaning

A computer cannot read meaning — it only works with numbers. Embedding models translate every paragraph into a unique list of numbers that captures its meaning as a location on a giant invisible map.

Raw document (PDF / text)	→	Chunking 200–500 words each	→	Embedding model chunk → vector	→	L2 Normalise magnitude = 1.0	→	Store in vector database
------------------------------	---	--------------------------------	---	-----------------------------------	---	------------------------------	---	--------------------------

Why These Steps Matter

Step	Why It Is Needed
Chunking	One big blob = one blurry average of everything. Chunks keep ideas separate and findable.
Embedding	Converts meaning into numbers the computer can compare mathematically.
L2 Normalisation	Makes all vectors the same length so comparisons are fair and fast.

■ IMAGE PROMPT

Soft pastel: long document scroll cut by scissors into chunks; each chunk through glowing 'embedding model' box; output as row of numbered bars (vector); all vectors stored in neat grid labeled 'vector database'; mint and peach; cream background; minimal text.

Experiment 2: Semantic Search

■ AIM

To implement a vector similarity search engine using cosine similarity as the ranking metric, enabling natural language queries to retrieve the most contextually relevant document chunks from a vector index — bypassing keyword matching entirely — and compare the performance trade-offs between brute-force Flat Index search and Approximate Nearest Neighbor indexing using HNSW graphs.

■ THEORY

Keyword Search vs Semantic Search

	Keyword Search	Semantic Search
Matches on	Exact same words	Meaning and concept
'car' finds	Only 'car'	'vehicle', 'automobile', 'sedan' too

Fails when	Different words, same idea	Almost never
Method	String comparison	Cosine similarity of vectors

The Search Flow



Flat Index vs HNSW — Speed Trade-off

Index Type	Method	Speed	Accuracy	Use When
Flat Index	Check every single vector one by one	Slow (millions = minutes)	100%	Small datasets
HNSW Graph	Navigate a smart graph, skip irrelevant zones	Fast (millions = milliseconds)	~99%	Production systems

■ IMAGE PROMPT

Soft pastel: user with question bubble → glowing embedding box → vector arrow into cluster of dots with one glowing nearest match; angle diagram labeled 'cosine similarity'; below: slow turtle (flat index) vs fast rocket (HNSW graph); light blue mint; cream background.

Experiment 3: Building a Simple RAG Pipeline

■ AIM

To construct a complete end-to-end Retrieval-Augmented Generation system that intercepts user queries, executes semantic search against a live vector database, injects the top retrieved chunks as grounded context into a structured LLM prompt, and produces factually verifiable answers — directly solving the twin limitations of knowledge cutoff dates and hallucination in standalone language models.

■ THEORY

The Two Problems RAG Fixes

LLM Problem	What Happens	RAG Fix
Knowledge cutoff	AI trained in 2023 has no idea about 2025 events	Vector DB is updated; AI reads fresh facts before answering
Hallucination	AI confidently invents facts it doesn't know	AI is forced to answer only from retrieved real documents

The RAG Orchestration Loop



Advanced RAG Upgrades

Upgrade	What It Does
Cross-Encoder Reranking	Double-checks retrieved chunks and reorders by true relevance before injection
Query Expansion	Rewrites a vague user question into a better optimised search query first

■ IMAGE PROMPT

Soft pastel circular loop: 5 steps connected by arrows — user question, semantic search into DB cylinder, top chunks glowing, chunks + question into prompt box, robot outputting grounded answer with green checkmark; broken chain icon 'no hallucination'; calendar icon 'no cutoff'; lavender peach mint; cream background.

Experiment 4: Fine-Tuning vs Prompt Engineering

■ AIM

To systematically analyze and compare the technical trade-offs, deployment costs, use case boundaries, and performance characteristics of context-based optimization via Prompt Engineering and RAG against parametric optimization via supervised Fine-Tuning — including parameter-efficient LoRA adaptation — producing a clear decision framework for selecting the correct strategy given a specific enterprise requirement.

■ THEORY

Two Ways to Make an AI Better

	RAG / Prompt Engineering	Fine-Tuning
What changes	The context (what AI reads)	The neural weights (AI's brain)
Best for	Changing facts, live data	Fixed style, format, custom tone
Speed to deploy	Hours	Days to weeks
Cost	Per-query token cost	Large upfront GPU cost
Learns new facts?	Yes — update the DB	No — facts go stale fast
Learns new style?	Inconsistent	Deeply and reliably

LoRA — Fine-Tuning on a Budget

Full fine-tuning retrains billions of weights — extremely expensive. LoRA freezes the entire base model and trains only a tiny adapter layer on top. Like pinning a sticky note onto a textbook instead of reprinting the whole book.

Method	Weights Trained	GPU Cost
Full Fine-Tuning	7,000,000,000+	Extremely high
LoRA Adapter	~1,000,000	Accessible on a single GPU

Decision Flow — Which to Choose?

Do facts change often?	→	YES → Use RAG update the DB	→	NO → Need a custom style?	→	YES → Fine-Tune (LoRA)	→	Need both? Fine-Tune style + RAG for facts
------------------------	---	-----------------------------	---	---------------------------	---	------------------------	---	--

■ IMAGE PROMPT

Soft pastel split two halves: left 'inject knowledge' — robot handed facts at query time, fast clock icon; right 'bake in skills' — robot in training chair with electric bolts absorbing style; centre decision tree (changing facts → left, fixed style → right); sticky note on textbook labeled LoRA; peach mint lavender; cream background; minimal text.